

Day3

울산 4반 이름 : 박인우

Chapter 1. 개인 정리 요약

순수 자바에서는 직접 `new ...` 등을 만들어 객체간 연결을 직접 해야함. 하지만 `spring boot` 는 `@`를 이용하여 객체간 연결을 자동으로 해준다.

명시적으로 "A를 만들고, B에 넣고, C를 실행해라"라고 쓰는 대신, 각 클래스에 붙은 어노테이션(`@RestController`, `@Service` 등)이 "나는 이런 역할을 하는 클래스다"라고 선언만 하면, `Spring`이 컨테이너 실행 시점에 알아서 찾아서 연결

내 이해 : `Spring Boot`에서는 `@`로 클래스마다 역할을 부여하고, 어느 정도 정해진 틀이나 템플릿을 가져와 메서드 등을 조금 수정하면 웹서비스를 쉽게 만들 수 있다?

→ `Spring Boot`는 웹서버, `HTTP` 요청 처리, 객체 생성·`DI`, `DB` 연결 같은 공통 기반과 정해진 사용 규칙을 미리 제공하고, 개발자는 `@`를 통해 각 클래스와 메서드의 역할·연결 관계를 선언한 뒤 실제 서비스에 필요한 비즈니스 로직을 작성한다.

→ `Spring Boot` = 웹 개발에서 반복되는 공통 구조와 실행 기반은 `Framework`가 맡고, 개발자는 `@`로 자신의 코드를 그 구조에 연결한 뒤 서비스 고유의 로직에 집중하게 해주는 프레임워크.

Chapter 2. 수업내용 정리

`@`붙이는거 유심히 보기 (컴퓨터가 보라는 주석), `@`를 보고 `Spring`이 자동으로 동작을 만들어줌, 이 코드를 어떻게 다뤄야 하는지 알려주는 지시표

이 @가 어디에 붙었는가? Spring에게 무슨 역할을 알려주는가? 이 @ 때문에 Spring이 자동으로 무엇을 해주는가?

@붙음 → Spring이 읽음 → 해당 클래스/메서드의 역할을 파악 → 필요한 객체 생성·연결·HTTP 처리 등을 자동 수행

solid원칙(역할과 책임을 구분하기 위함)

S : 단일 책임 원칙 : 하나의 클래스는 하나의 책임만 가짐 : **controller, service, repository** 등 계층 분리

O : 개방-폐쇄 원칙 : 기존 코드는 수정하지 않고 기능을 확장할 수 있어야 한다 : 인터페이스 기반 구현체 추가, 전략 패턴 활용

L : 리스코프 치환 원칙 : 부모 타입 대신 자식 객체를 사용해도 동일하게 동작해야 한다 : 인터페이스 구현체 교체 가능

I : 인터페이스 분리 원칙 : 필요한 기능만 가진 작은 인터페이스로 분리한다. : 역할별 인터페이스 설계(작고 구체적인 인터페이스 여러개)

D : 의존성 역전 원칙 : 구현 클래스가 아닌 추상화(인터페이스)에 의존한다 : **Spring**의 **DI** 와 **IoC**의 핵심 원칙(객체 연결을 인터페이스로)

객체 생성 패턴 : 왜 패턴이 필요한가?: 코드 중복, 의존성 증가, 테스트 어려움

싱글톤 : 객체를 하나만 생성 : 매번 **new**선언을 해서 객체 생성해야됨

팩토리 패턴 : **new**를 쓰지않고, 무엇을 만들지를 공장(**factor**)에 맡기는 방식 : 결합도 감소, 확장 용이, 다형성 활용 : 인터페이스, **factory**클래스+**if else**문

빌더 패턴 : 필드가 많거나 선택적 값이 많은 객체를 안전하게 생성하기 위한 패턴 : 매개변수 순서 안헛갈리게

객체 지향 **srp, ocp/dip** : 클래스는 단 하나의 책임만.

스프링부트 : **di**컨테이너 , **auto configuration** , **@service/@repository**

Rest API : **@RestController**, HTTP Method/status, postman

절차기반(함수중심 - 데이터와 로직이 분리, 수정 시 전체 흐름에 영향), 객체 지향(데이터와 동작이 묶임, 책임분리)

스프링 부트 : 자동설정, **di**컨테이너 = 나눠진 객체들을 자동으로 연결해주는 도구(설정 자동화, 톰캣, 의존성, **@**)

파일 분리 후 수동 연결 코드가 많아짐 -> 스프링이 자동으로 객체 생성해줌(**main()**에서 **new**로 수동으로 생성/주입하던걸 **@service/@autowired** 로 주입)

빈(클래스를 가지고 만든 객체(목적성을 가지고있음)) -> **IoC**컨테이너(객체들을 관리하는 하나의 컨테이너 박스), 직접 객체를 생성하는 경우는 잘 없음(객체를 생성하는 부분이 없음), 스캐너가 모든 클래스 파일 탐색(**@**붙은거 찾음)->객체 자동 생성->**IoC**컨테이너에 담음, 설정한**@**에 따라 객체의 역할들이 부여됨, **IoC**컨테이너 안에 클래스들은 언제든지,어디든 마음대로 꺼내 쓸 수 있음, 객체들간의 값을 주고받는데에 쉬워짐, **@**은 언제 객체를 생성되는지, 역할은 무엇인지, 애가 가지고 있는 메서드는 언제 호출되는지 정도만 꼭 파악

객체가 다른 구체 객체를 직접 만들고 의존하면 독립성이 깨질 수 있다. 그래서 **Spring**은 객체 생성·연결을 **IoC Container**에 넘기고, 인터페이스를 사이에 두는 **DI** 방식으로 결합도를 낮춘다.

-**config** : 설정 클래스 : 애플리케이션이 어떤 설정으로 동작하는지(**db**연결, 포트 등)설정, **application.yml**과 **config**패키지를 함께 다룸

-**domain** : 도메인 객체 : 데이터의 형태 : **@Entity**

-**exception** : 예외처리

-**repository(dao)** : 실제 DB접근 계층

-**security** : 보안(인증/인가를 어떻게 처리하는지)

-**service** : 비즈니스 로직 계층(인증 처리 등)

-**web(controller)** : HTTP 요청을 받는 계층 : 창구

-LoginAuthApplication.java : main()이 있는 프로그램 시작점, 톰캣을 띄우며 실행 :
@SpringBootApplication

-resource/static : html 등 실제 사용자가 보는 화면, 컨트롤러와 연결

교수님 코드 구조

Spring Boot 시작 → @SpringBootApplication → Bean 생성 + DI + 설정 →

브라우저 HTTP 요청 → Spring Security, JwtAuthenticationFilter → Controller "HTTP를 받는다" →
Service "무슨 일을 할지 결정한다" → Repository "DB와 통신한다" → MariaDB → Repository →
Service → Controller → JSON / Cookie / Redirect → 브라우저

개발자가 @으로 “이 객체의 역할, 필요한 의존성, 어떤 요청에 반응할지”를 선언하면, Spring Boot가
객체 생성·DI·HTTP 연결·보안·설정·검증·트랜잭션·예외 처리 같은 공통 실행 구조를 대신 관리하고,
개발자는 그 안에서 실제 회원가입·로그인 같은 비즈니스 로직의 순서를 작성한다.

Chapter 3. 주요 어노테이션 및 역할

1. @SpringBootApplication

→ 애플리케이션 시작 + Component Scan

2. @Controller / @Service / @Repository / @Component

→ 객체를 Spring Bean으로 관리

3. 생성자 DI

→ 필요한 Bean들을 서로 자동 연결

4. @GetMapping / @PostMapping

→ HTTP 요청을 Java 메서드와 연결

5. Spring Security Filter Chain

→ Controller 전에 인증/인가 처리

6. @Configuration / @Bean

→ Spring이 사용할 객체와 설정 등록

7. @ConfigurationProperties / @Value

→ application.yml 값을 Java 객체에 연결

8. @Valid

→ 요청 데이터 검증 실행

9. @Transactional

→ DB 작업의 트랜잭션 관리

10. @RestControllerAdvice / @ExceptionHandler

→ 예외를 HTTP 오류 응답으로 변환

Chapter 4. 코드 구조 이해(실습교수님)

User.java

User는 사용자 id와 암호화된 비밀번호를 보관하며, 외부에서 직접 생성하지 못하게 하고 `User.of()`를 통해 반드시 BCrypt 암호화를 거쳐 객체를 생성하며, 로그인 시 `matchPw()`로 입력 비밀번호와 저장된 해시가 일치하는지 검사하도록 만든 순수 Java 사용자 객체

TokenProvider.java

TokenProvider는 @Component로 Spring Bean으로 등록되는 JWT 생성 전용 객체이며, 로그인 성공 후 전달받은 `userId`를 JWT의 `subject`로 넣고 30분의 만료시간을 설정한 뒤 `Secret Key`로 서명하여 JWT 문자열을 반환한다. Spring은 이 객체의 생성과 DI를 담당하고, JWT의 내용·만료시간·생성 순서는 개발자가 `generate()` 안에 직접 정의한다.

AuthService.java

AuthService는 @Service로 Spring Bean이 되어 회원가입·로그인 비즈니스 로직을 담당하고, @Autowired 생성자 DI를 통해 @Component인 TokenProvider를 받아 사용한다. 회원가입에서는 `User.of()`로 BCrypt 처리된 사용자를 HashMap에 저장하고, 로그인에서는 사용자를 조회해

`User.matchPw()`로 비밀번호를 검사한 뒤 성공하면 `TokenProvider.generate()`를 호출해 JWT를 반환한다.

AuthController.java

`AuthController`는 `@RestController`로 Spring이 관리하는 REST API Bean이며, `@RequestMapping`과 `@PostMapping`으로 `/api/auth/register`, `/api/auth/login` HTTP 요청을 Java 메서드와 연결한다. `@RequestBody`는 클라이언트의 JSON을 `AuthRequest` 객체로 변환하고, `@Autowired`는 `@Service`인 `AuthService`를 주입한다. Controller는 실제 회원가입·인증 로직은 Service에 맡기고, 결과를 `ResponseEntity`와 `TokenResponse`를 이용해 HTTP 상태 코드와 JSON 형태로 클라이언트에 반환한다.

Application.java

`Application`은 `@SpringBootApplication`이 붙은 Spring Boot 시작 클래스이며, `main()`에서 `SpringApplication.run()`을 호출하면 Spring이 `login_server` 패키지 아래의 `@Component`·`@Service`·`@RestController` 등을 찾아 Bean으로 만들고 DI로 연결하며, 웹 관련 자동 설정과 내장 Tomcat을 실행해 HTTP 요청을 받을 준비를 한다.

Http파일

POST(or GET) = 요청 종류, URL = 요청 주소, Spring은 `@PostMapping` / `@GetMapping`으로 그 요청을 특정 메서드와 연결한다.

최종 흐름

`@SpringBootApplication` → Spring Boot 시작 → `@ComponentScan` → `@Component` / `@Service` / `@RestController` 탐색 → Bean 생성 + IoC Container 등록 → `@Autowired` → Bean끼리 DI로 연결 → Tomcat 실행 → HTTP 요청 대기

요청이 오면

app.http POST `/api/auth/login` → `@RestController` `@RequestMapping("/api/auth")` `@PostMapping("/login")` → `AuthController.login()` → `@RequestBody` JSON → `AuthRequest` → `@Autowired`로 주입된 `AuthService` → `@Service` 회원 조회 + 비밀번호 검사 → `@Autowired`로 주입된 `TokenProvider` → `@Component` JWT 생성 → Controller로 반환 → `TokenResponse` →

@RestController가 JSON 응답으로 변환

Chapter 5. 궁금했던 사항 및 보충

빌더 패턴?

객체를 만들 때 생성자에 값이 너무 많아지는 문제

복잡한 객체 생성을 읽기 쉽고 유연하게 만드는 객체 생성 패턴

SOLID의 D. : 의존성 역전 원칙

상위 로직이 구체적인 구현체에 직접 의존하지 말고 추상화(interface)에 의존하자.

인터페이스 같은거 이용!

톰캣?

Tomcat = 외부의 HTTP 요청을 받아 Spring에게 넘겨주는 Java 웹서버 실행 환경

@Value — application.yml의 값을 필드에 하나씩 꺼내 주입.

@Autowired — 필요한 빈을 스프링이 찾아서 자동으로 연결(주입)

@RequiredArgsConstructor(Lombok) — final 필드 기준으로 생성자를 자동 생성해서, 생성자 주입을 간편하게

스프링부트가 객체간 연결이 정해진게 있어서 로직대로 해준다면, 각 객체별로도 정해진 틀(매개변수라던가 타입설정 같은거..)이 있나?

정해져 있는 경우가 꽤 많다. Spring이 제공하는 기능을 쓸수록 “이 자리에 이런 타입/매개변수를 쓰면 Spring이 이렇게 처리한다”는 약속이 많아짐

라이브러리는 보통:

내가 필요할 때 라이브러리 함수를 호출한다.

반면 **Spring** 같은 프레임워크는:

프레임워크가 전체 골격과 실행 흐름을 가지고 있고, 개발자가 정해진 위치와 규칙에 맞춰 자기 코드를 끼워 넣는다.

DI한다?

어떤 객체가 필요한 다른 객체를 직접 만들지 않고, 외부(**Spring**)에서 받아 쓰는 것

@마다 객체 생성 하는것 뿐만 아니라 추가 기능도 달려있는 경우가 있다. 어떠한 역할을 하는**@**인지 파악하자

Rest API : 웹서버는 필요한 정보만 보내고 다른 무거운 작업들 사용자의 웹브라우저에서 실행되게 하자
→ 서버와 클라이언트간에는 데이터만 주고받음.(json으로)